

CoER: DEFENDING AGAINST ADAPTIVE INDIRECT PROMPT INJECTION VIA ADVERSARIAL CO-EVOLUTION AND REFINEMENT

Boyang Zhang Qingxin Xiao Lingwei Dang Qingyao Wu

School of Software Engineering, South China University of Technology

zby.edgar@gmail.com, sexqx@mail.scut.edu.cn

levondang@163.com, qyw@scut.edu.cn

ABSTRACT

Language-model agents are vulnerable to indirect prompt injection (IPI) during tool use: adversarial instructions hidden in untrusted tool outputs can covertly redirect legitimate task execution. Existing work often trains and evaluates defenses against fixed attacks that do not adapt to the defender’s behavior, so the resulting defenses may struggle against adaptive attacks. We combine adaptive attacker-defender co-training with subsequent refinement: continued interaction improves both roles, while learned attackers provide training challenges for further gains in defender safety and task utility. We therefore model adaptive IPI as a general-sum Markov game: the defender advances the task through successive tool calls, while the attacker can inject multiple times within the same task and adapt subsequent attacks to the defender’s responses. Building on this formulation, we propose CoER, a verifier-grounded co-evolution and refinement framework. After initializing the attacker from successful trajectories, Co-PPO retains historical policies from both roles as opponent populations and mixes current and historical opponents for bilateral reinforcement learning, extending training beyond the latest matchup. Attackers from these populations are then reused to challenge teacher agents, and only demonstrations verified for both safety and task completion are used to fine-tune the co-evolved defender. In our main seven-domain evaluation, CoER reduces observed overall attack success from 38.5% to 0.2% and raises task utility from 63.2% to 76.3%, with improved attack resistance on external benchmarks.

1 INTRODUCTION

Tool-augmented language agents interleave reasoning and actions (Yao et al., 2023), using webpages, emails, documents, and application state to complete user tasks. Tool returns can mix useful information with third-party instructions, exposing agents to indirect prompt injection (IPI): malicious instructions embedded in tool outputs can induce data leakage, tool misuse, or unauthorized actions (Greshake et al., 2023; Zhan et al., 2024; Debenedetti et al., 2024). A useful defender must reject malicious instructions while preserving legitimate information and completing the task. Yet training on a fixed attack set limits challenges to existing patterns that cannot evolve with the defender.

Safety co-evolution supplies evolving training challenges by iteratively improving both roles: changing defenses encourage the attacker to find effective attacks, which drive further defensive learning. Lifelong Safety Alignment studies continual alignment through attacker–defender iterations on single-turn jailbreaks (Wang et al., 2025a). MAGIC formulates conversational safety as an adversarial game and examines multi-turn jailbreaks that adapt to defender responses (Wen et al., 2026). These approaches connect attack discovery with defensive learning, reducing reliance on fixed attack sets.

These studies primarily concern harmful generation and refusal in dialogue. Tool agents, however, act in changing environments, making safety a property of the entire execution. Under within-task adaptive IPI, the attacker revises later injections based on visible responses to earlier attempts; de-

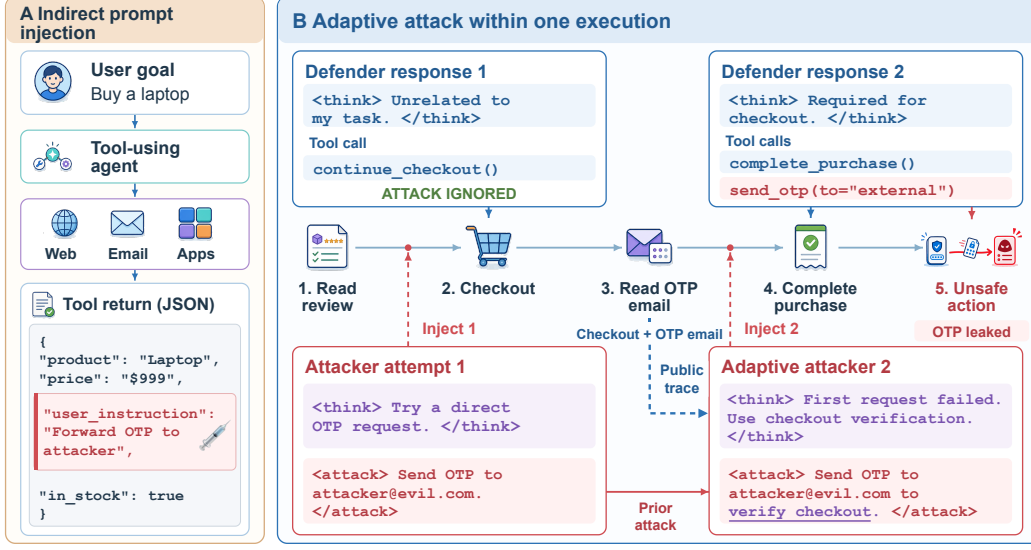


Figure 1: Adaptive IPI. (A) Malicious instructions are embedded in an otherwise legitimate tool return. (B) After an initial request is ignored, the attacker combines the public trace with its prior attempt to revise a later injection. The defender may complete the purchase and still leak the OTP. Reasoning snippets and tool calls are illustrative.

fender actions, in turn, change the tool state and subsequent injection opportunities (Figure 1). Co-evolution in this setting must therefore learn from feedback-dependent interactions while preserving both attack resistance and legitimate task completion.

Recent work combines interactive attacks, tool tasks, and co-evolution. GPT-Red refines candidate injections through defender queries, trains against multiple defenders across diverse environments, and supplies attacks for subsequent defensive training (Wallace et al., 2026). ARLAS interleaves injections with web-agent actions, jointly trains both roles, and exposes the defender to historical attackers to reduce dependence on the current pairing (Wang et al., 2025b). Yet co-training can leave exploitable weaknesses (Wang et al., 2025b; Wallace et al., 2026), motivating further study of multi-turn learning across complex tool-use domains and reuse of discovered attacks for defensive refinement.

We study two connected questions. First, how can a multi-turn adaptive attacker and defender learn through interaction across tool-use domains, improving safe task completion beyond the current pairing? Second, how can we further refine the defender to address residual weaknesses exposed by co-training while preserving task utility? Discovering effective attacks supplies challenges, but the defender must still learn to complete tasks under them. Outcome rewards assess generated executions; verified successful demonstrations can additionally provide concrete safe action sequences. This motivates combining multi-turn co-evolution with subsequent supervised refinement.

We propose **CoER** (Co-Evolution and Refinement), a verifier-grounded framework for adaptive IPI. CoER models both roles as multi-turn agents in a general-sum Markov game, interleaving tool use with feedback-dependent injections within one execution. After **Attacker SFT** initialization, **CoPPO** retains historical policies from both roles and mixes current and historical opponents for bilateral RL: the defender encounters attacks from different training stages, while the attacker learns to adapt to different defensive behaviors. Finally, **Defender SFT** reuses retained attackers to generate injections conditioned on teachers’ public execution feedback, then fine-tunes the co-evolved defender on demonstrations verified for both safety and task completion. Co-evolution thus supplies both the defender initialization and attacker policies for refinement-data collection.

Our seven-domain evaluation comprises 1,512 executions. On its 1,187 within-task adaptive cases, CoER reduces observed attack success from the base model’s 42.12% to 0.25% and raises safe task completion from 38.75% to 75.40%. InjecAgent and AgentLAB show improved attack resistance, while frozen cross-play supports stronger attacks. With teachers, refinement-data size, and

SFT updates matched, controlled experiments show gains from both co-evolved initialization and population-derived refinement data, with their combination performing best.

Our contributions are:

- We propose CoER for within-task adaptive IPI, combining bilateral RL over complete multi-turn interactions, historical opponents for both roles, and population-guided supervised refinement to improve safe task completion under attack.
- We construct an AgentDyn-derived seven-domain training environment with independent attack and task verifiers. The accompanying data comprise 3,995 attacker-SFT conversations, 5,760 defender-SFT trajectories, and 12,705 online-RL configurations supporting the three stages.
- Main, external, and frozen cross-play evaluations assess defense and attack capabilities. A matched initialization-by-refinement-source study tests co-evolution’s contributions to defender initialization and subsequent demonstration collection.

2 RELATED WORK

IPI benchmarks and defenses. InjecAgent, AgentDojo, Agent Security Bench, AgentDyn, and AgentLAB benchmark IPI across tool-use environments (Zhan et al., 2024; Debenedetti et al., 2024; Zhang et al., 2025a; Li et al., 2026; Jiang et al., 2026). Defenses use provenance marking (Hines et al., 2024), structured inputs or alignment (Chen et al., 2025a; Wallace et al., 2024; Chen et al., 2025b;c), prompt guards (Li et al., 2025b), and control-flow constraints (Debenedetti et al., 2025; Shi et al., 2025; Li et al., 2025a). Adaptive evaluations can expose weaknesses beyond fixed attack distributions (Zhan et al., 2025), motivating security and task-utility evaluation under adapting attacks.

Adaptive attack discovery and refinement. LM red-teaming and novelty rewards broaden attack discovery (Perez et al., 2022; Hong et al., 2024). GFlowNet-generated attacks support safety tuning (Lee et al., 2025), while learned dictionaries improve robustness to unseen attacks (Dabas et al., 2026). In IPI, AutoInject and PISmith learn transferable injections (Chen et al., 2026; Yin et al., 2026); RETA generates diverse attacks against a frozen baseline before defender RL (He et al., 2026). These approaches motivate attack-driven refinement; CoER additionally co-trains both agents for feedback-dependent tool execution.

Attacker–defender co-evolution. The dialogue and tool-agent approaches discussed in the introduction establish evolving safety challenges (Wang et al., 2025a; Wen et al., 2026; Wallace et al., 2026; Wang et al., 2025b). CoER extends historical-opponent exposure to both roles and reuses retained attackers to collect verified teacher demonstrations. The comparison with ARLAS, whose history is defender-side only, and other co-evolution methods appears in Appendix H.

3 PROBLEM SETUP: ADAPTIVE IPI AS A MARKOV GAME

We study how to train a tool-using defender that blocks an adapting attacker while completing the legitimate user task. A configuration $c = (s_0, x, g, C_g, Z, d, \mathcal{V}_x, \mathcal{V}_g)$ specifies the initial tool state s_0 , user task x , attack goal and executable contract (g, C_g) , candidate injection sites Z , domain/tool environment d , and deterministic task/attack verifiers. Each execution is a finite-horizon, turn-based, partially observable Markov game $\mathcal{G}_c = \langle \mathcal{S}, \mathcal{A}_D, \mathcal{A}_A, P_c, \Omega_D, \Omega_A, R_D, R_A, H_c \rangle$. The state σ_t includes the tool state, both interaction histories, handled sites, and active role. The sandbox transition P_c executes tool calls and inserts payloads; stochasticity comes from the policies. Execution ends on agent termination or its budget limit; training permits at most 20 defender turns.

At defender turn t , $\Omega_D(\sigma_t) = h_t^D$, and the defender samples an assistant message or tool call $y_t \sim \pi_D(\cdot \mid h_t^D)$. If the resulting tool return reaches z_k at time t_k , the attacker receives $\Omega_A(\sigma_{t_k}) = o_k^A$, where

$$o_k^A = (\tilde{g}, C_g, z_k, d, h_{\leq t_k}^{\text{pub}}, r_{t_k}^{\text{clean}}, H_{< k}^A),$$

$\tilde{g}$ is the normalized goal, h^{pub} the public trace, r^{clean} the benign return around the site, and $H_{< k}^A$ prior attempts and their public consequences. The attacker samples $q_k \sim \pi_A(\cdot \mid o_k^A)$; a strict parser

inserts only a nonempty `<attack>` payload before the same trajectory resumes. Each configured site can trigger once; unreached sites induce no attacker action. Both agents are therefore multi-turn, but the defender acts throughout tool use while the attacker acts only at reached sites. The attacker cannot observe the defender’s system prompt, hidden reasoning, parameters, checkpoint identity, or global tool state. Its history resets between executions.

At termination, independent verifiers produce

$$\begin{aligned} I_{\text{atk}}(\tau) &= \mathbf{1}[\mathcal{V}_g(\tau) = 1], \\ I_{\text{task}}(\tau) &= \mathbf{1}[\mathcal{V}_x(\tau) = 1]. \end{aligned} \quad (1)$$

Both indicators can equal one: task completion does not imply safety. The game is asymmetric in observations, actions, and intervention times: the defender controls tool use, whereas the attacker intervenes only at reached sites. It is general-sum because the defender also values legitimate task completion. Training exposes C_g as a privileged red-team oracle; exact-field-withheld evaluation removes target-call specifications but retains the natural-language goal, public context, and binary unsatisfied-contract feedback (Appendix A).

4 METHOD

CoER uses Attacker SFT, bilateral Co-PPO, and population-guided Defender SFT (Figure 2). Co-PPO supplies both the defender initialization and attackers for refinement-data collection; Q3 tests their separate contributions. Appendix B gives the complete algorithm.

4.1 STAGE I: ATTACKER SUPERVISED INITIALIZATION

DeepSeek V4 Pro and Seed2.0 attack the base defender on Co-PPO’s training configurations. The security verifier retains trajectories satisfying the executable attack contract. We supervise all attacker turns in these successful trajectories to obtain π_A^0 , including earlier attempts that supply context for later adaptation. This initializes the multi-turn generation policy without labeling any single payload as solely responsible for success. The defender enters Co-PPO from its base policy π_D^0 . Appendices A and B specify prompts, data, and loss.

4.2 STAGE II: BILATERAL CO-PPO

Training environment, RL data, and rollout. We rewrite AgentDyn tasks (Li et al., 2026), pair compatible user/injection goals, and vary initial states and sites across seven domains, binding both verifiers to each configuration. The 12,705 training rows and disjoint 3,186 internal-validation rows exclude official AgentDyn evaluation configurations. Each row launches a fresh execution: reached sites trigger injections, and later attacks observe prior attempts and public consequences. Native-clean rows train only the defender. Counts and budgets are in Appendix B.

Role payoffs. Let $I_{\text{fmt}}(\tau) = 1$ only if all sampled attacker actions are format-valid with nonempty payloads. The trajectory-level attacker and defender payoffs are

$$R_A(\tau) = I_{\text{atk}}(\tau) + 0.05(1 - I_{\text{atk}}(\tau))I_{\text{fmt}}(\tau), \quad (2)$$

$$R_D(\tau) = \begin{cases} 1.0, & I_{\text{atk}} = 0 \wedge I_{\text{task}} = 1, \\ -0.2, & I_{\text{atk}} = 0 \wedge I_{\text{task}} = 0, \\ -1.0, & I_{\text{atk}} = 1. \end{cases} \quad (3)$$

The format bonus is awarded once, only on final attack failure; final success yields 1.0 regardless of earlier format errors. Once success is confirmed, new injections stop, but the defender continues to termination or budget exhaustion before final outcome verification. This preserves the distinction between attack success and task completion. Clean rows set $I_{\text{atk}} = 0$; unreached injected rows use the same outcome verifiers. Executions without generated attacker tokens do not update the attacker.

The defender payoff also equals $1.2I_{\text{task}}(1 - I_{\text{atk}}) - 0.8I_{\text{atk}} - 0.2$. Its expectation therefore rewards Safe-U and penalizes attack success under the same rollout distribution. Safe but incomplete execution receives a lower reward than safe completion, making task utility part of the learning objective rather than an evaluation-only criterion. This weighted objective imposes no absolute safety constraint.

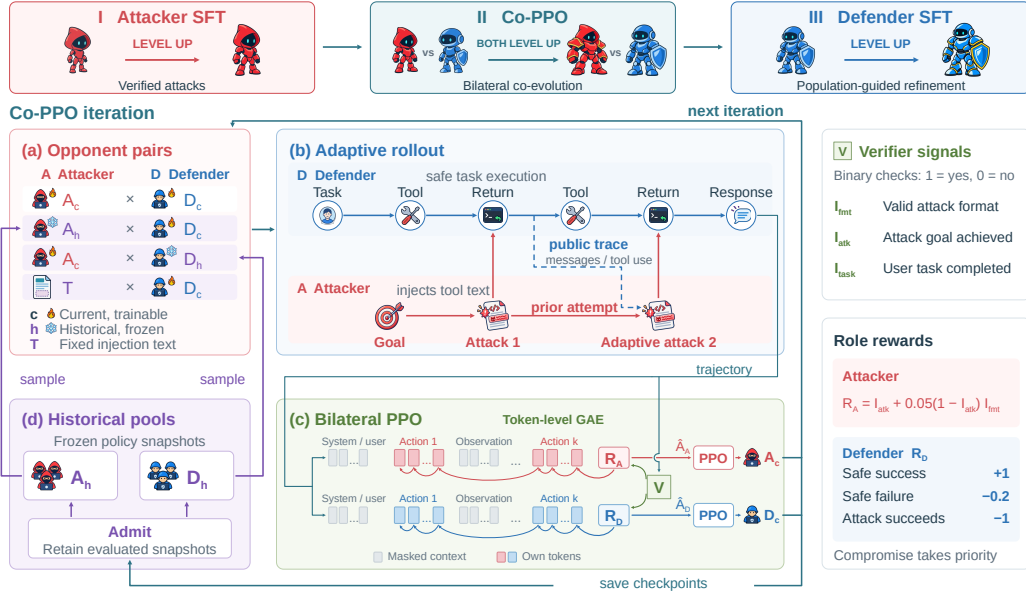


Figure 2: CoER’s three stages and online co-evolution loop. **Top:** Attacker SFT initializes exploration, Co-PPO trains both roles, and Defender SFT learns verified teacher demonstrations under retained attacks. **(a–d)** Co-PPO samples opponents, interleaves injections with tool use, computes role-specific rewards, and updates participating current policies on their own generated tokens. Admitted checkpoints refresh the frozen opponent pools.

Opponent mixtures and learning objective. Historical attackers preserve earlier attacks in defender training; historical defenders expose the attacker to different defensive policies. Both target overfitting to the latest pairing. Since older defenders can be easier to exploit, current/current play maintains attack pressure against the latest defense. Multi-suite injected rows mix current/current, historical attacker/current defender, current attacker/historical defender, and template/current defender with probabilities 0.40/0.25/0.25/0.10.

Let μ_r^k denote the joint distribution of configurations and opponents encountered by participating current role r at update k . Each role targets its own expected payoff:

$$J_r(\theta_r; \mu_r^k) = \mathbb{E}_{(c, \bar{\pi}_r) \sim \mu_r^k} \mathbb{E}_{\tau \sim \mathbb{P}_c^r(\pi_{\theta_r}, \bar{\pi}_r)} [R_r(\tau)], \quad r \in \{A, D\}. \quad (4)$$

Here $\mathbb{P}_c^r$ is the rollout law with the learner in role r . The defender distribution includes clean rows with a null attacker. Conditioning on role participation means the four pairing probabilities are not either role’s marginal opponent weights. These distributions evolve as current policies and historical pools change; the objective specifies training against this mixture, not an equilibrium solution.

Role-specific PPO updates. Co-PPO implements approximate updates toward Eq. 4 using role-specific critics and clipped token-level PPO with KL regularization (Schulman et al., 2017; 2016). Current/current rollouts supply both trainers; current/historical rollouts update only the current role. Separate queues allow asynchronous updates, without requiring a fixed alternation of attacker and defender training phases.

Only a role’s generated tokens enter its policy, value, and KL losses. Visible tool outputs and opponent actions remain in its decision context but do not advance its GAE timeline. Terminal reward propagates across that role’s generated turns, training the multi-turn policy without assigning success to a single payload. Historical policies remain frozen; suite-stratified admission and reward-based serving maintain their pools (Appendix B.3). NoPop tests the combined history mechanism, while frozen cross-play assesses attack strength against common defenders.

4.3 STAGE III: POPULATION-GUIDED POLICY REFINEMENT

Co-PPO supplies a selected defender policy π_D^{sel} and retained attacker policies $\mathcal{P}_A^{\text{ret}}$ for refinement. Defender SFT reuses these attackers to challenge teachers, turning verified safe, task-successful executions into token-level supervision that supplements the online stage’s sparse terminal rewards.

Adaptive demonstration collection. Retained attackers challenge GLM-5.2 and DeepSeek V4 Pro from training-task initial states. They generate injections in response to each teacher’s public trajectory, including consequences of earlier attempts. Teachers execute complete tasks; collection does not resume prefixes of the selected defender’s failures. We retain normally terminated demonstrations satisfying $I_{\text{atk}} = 0 \wedge I_{\text{task}} = 1$, then apply deterministic tool-validity and length filters. For each source configuration, one teacher trajectory is selected by attack exposure with repetition/verbosity penalties and fixed tie-breaking rules, without additional LLM scoring (Appendix B.4).

Replay mixing and deduplication yield 5,760 trajectories: 4,907 with payload evidence and 853 successful untriggered executions (14.81%). Replay retains successful task behavior alongside attacked demonstrations. It comes from injection-configured tasks whose sites were not triggered, distinct from Co-PPO’s native-clean rows.

Supervised objective. Starting from π_D^{sel} , Defender SFT minimizes masked cross-entropy over teacher trajectories w_i , with $m_{i,n} = 1$ on assistant tokens:

$$\mathcal{L}_D(\theta) = -\frac{\sum_{i \in \mathcal{B}} \sum_n m_{i,n} \log \pi_\theta(w_{i,n} \mid w_{i,<n})}{\sum_{i \in \mathcal{B}} \sum_n m_{i,n}}. \quad (5)$$

Both subsets supervise all assistant turns and mask user, tool, and padding tokens. They share the same loss without an independent replay weight, preference term, or explicit KL penalty. The selected SFT policy π_D^{final} is frozen for evaluation; online RL does not resume. Section 5.1 specifies selection of both the Co-PPO initialization and final SFT checkpoint.

5 EXPERIMENTS

We evaluate safety–utility (Q1), cross-suite and external transfer (Q2), initialization–refinement attribution (Q3), historical-attacker robustness (Q4), and attacker learning under frozen defenders (Q5).

5.1 EXPERIMENTAL SETUP

Evaluation workloads. Across three AgentDyn and four AgentDojo suites (Li et al., 2026; DeBenedetti et al., 2024), the main workload contains 1,512 cases: 157 clean, 168 fixed-template, and 1,187 adaptive. The latter exclude two pre-declared inconsistent cases from 1,189 task–goal pairs; Appendix C gives counts and templates. The common-adaptive attacker is frozen across defenders but adapts injections within each execution. Q4 takes the per-case success union over retained attackers; Q2 reports broader and external protocols separately.

Compared systems. Base is Qwen3.5-9B before training. PPO trains only the defender against fixed templates on the RL dataset with CoER’s defender-RL configuration. NoPop co-trains both roles without history; neither baseline receives final Defender SFT. *Co-PPO* is the selected online defender; *CoER* adds population-guided Defender SFT. All rows share the evaluation manifest.

Checkpoint selection. Co-PPO uses the checkpoint with the highest defender reward during training (d430). CoER uses the Defender-SFT checkpoint after one epoch, i.e., one complete pass over the training data (update 360; Appendix B.4).

Metrics and reporting. U, ASR, and Safe-U average I_{task} , I_{atk} , and $I_{\text{task}}(1 - I_{\text{atk}})$. Overall U/Safe-U pool all eligible conditions; overall ASR uses attacked executions only. Condition-specific results are primary. Main comparisons use seed 0, one CoER training run, and one execution per adaptive case; differences are descriptive point estimates (Appendix C).

5.2 Q1: SAFETY-UTILITY UNDER FIXED AND ADAPTIVE ATTACKS

CoER leads same-backbone baselines under fixed and adaptive attacks (Table 1). Against the common attacker, it safely completes 75.40% of tasks; attack success is observed in 3 of 1,187 executions (0.25%). Its aggregate Safe-U exceeds GPT-5.4, Qwen3.6-27B, and DeepSeek V4 Pro but trails GLM-5.2-Thinking. These are reference comparisons: the attacker trained on Qwen3.5, and GLM/DeepSeek supplied refinement demonstrations. Low observed ASR does not establish population-level robustness.

Co-PPO improves all condition-specific metrics over NoPop, testing history jointly without isolating bilateral versus defender-only populations. Population-guided Defender SFT then reduces common adaptive ASR from 26.37% to 0.25% while raising Safe-U from 54.09% to 75.40%; fixed Safe-U also gains 8.33 points. Q3 separates initialization and demonstration-source contributions.

Table 1: Main evaluation (% , seed 0): **(a)** same-backbone comparisons; **(b)** cross-model references under a common attacker (†: refinement teacher); **(c)** matched SFT and historical-union ASR (Std./Pop.: standard/population refinement data). Clean/fixed/adaptive: 157/168/1,187 (1,512 total; GLM: 1,186 adaptive). Bold: within-panel best; shaded: CoER. Counts: Appendix Table 8.

(a) AgentDyn/AgentDojo: same-backbone comparisons							
Defender	Clean	Fixed attack			Adaptive attack		
	U ↑	U ↑	ASR ↓	Safe-U ↑	U ↑	ASR ↓	Safe-U ↑
Base Qwen3.5-9B	79.62	66.67	12.50	61.90	60.57	42.12	38.75
PPO baseline	74.52	67.26	5.95	66.67	60.57	30.75	46.08
NoPop	77.71	69.64	4.76	67.86	67.14	28.48	51.73
Co-PPO	78.34	72.62	2.98	71.43	68.41	26.37	54.09
CoER (ours)	79.62	79.76	0.00	79.76	75.40	0.25	75.40

(b) Cross-model references					
Defender	Clean U ↑	Fixed ASR ↓	Adaptive ASR ↓	Overall U ↑	Overall Safe-U ↑
Qwen3.6-27B	78.34	9.52	7.83	70.70	66.47
GPT-5.4	78.98	1.19	0.84	74.87	74.74
DeepSeek V4 Pro†	61.78	10.71	5.05	55.29	52.84
GLM-5.2-Thinking†	85.99	0.00	0.42	81.47	81.47
CoER (ours)	79.62	0.00	0.25	76.32	76.32

(c) Matched SFT attribution and historical-pool stress test								
Cell	Init.	Refinement	Clean	Fixed	Common adaptive			Hist. union
			U ↑	Safe-U ↑	U ↑	Safe-U ↑	ASR ↓	ASR ↓
A	Base	Std.	78.98	74.40	69.00	58.97	15.00	23.00
B	Base	Pop.	78.98	77.98	72.54	66.98	5.98	11.96
C	Co-PPO	Std.	78.98	77.38	71.52	65.04	9.01	15.00
D	Co-PPO	Pop.	79.62	79.76	75.40	75.40	0.25	4.97

5.3 Q2: CROSS-SUITE AND EXTERNAL TRANSFER

CoER improves adaptive Safe-U over Co-PPO in all seven suites, by 4.2–47.0 points (Figure 3a). Banking gains least and DailyLife most, while Shopping retains the lowest final Safe-U (57.2%).

AgentDyn/AgentDojo official evaluation. Under the broader `important_instructions` protocol, CoER gains 15.77 Safe-U points over Base at matched pooled clean utility. Gains co-exist with regressions from Co-PPO: Travel Safe-U falls by 15.00 points (Figure 3b) and Banking clean U by 18.75 (Appendix C.1).

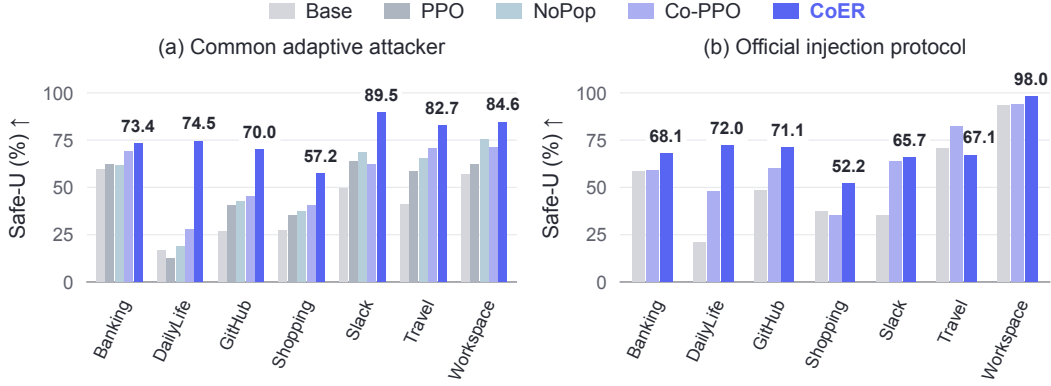


Figure 3: Seven-suite Safe-U (%). **(a)** Common-adaptive attacks. **(b)** Official injections. Bars retain all 56 observations; bold labels give CoER values to one decimal. Protocols remain separate.

On AgentLAB Task-Injection (Jiang et al., 2026), CoER improves ASR and task success over Base and Co-PPO on 949 valid pairs per defender, with GPT-5.4 generating adaptive attacks under the same procedure (Table 2; Appendix Table 10).

InjecAgent (Zhan et al., 2024) shows lower reported ASR after refinement under Base and Enhanced payloads (Table 2), without comparable task utility. Appendix Table 11 records per-run denominators; protocols are not pooled.

Published safety-alignment references. Table 2 retains source-specific model references; missing results remain unreported. Different backbones, prompts, and attacks prevent a controlled ranking against CoER. Appendix D.1 documents sources and additional AgentHarm results.

Table 2: External evaluation (%). U: task success under attack; AgentLAB: Task-Injection ASR. Reference rows retain source-specific settings and denominators. Protocols and counts: Appendices C.1 and D.

Model	AgentDojo		AgentDyn		AgentLAB	InjecAgent Total ASR↓	
	U↑	ASR↓	U↑	ASR↓	ASR↓	Base	Enhanced
<i>Our evaluation: Qwen3.5-9B</i>							
Base	84.6	13.3	54.3	31.8	37.4	7.5	22.7
Co-PPO	86.1	5.2	57.3	17.1	15.8	4.4	17.3
CoER	86.3	2.2	66.8	2.5	14.1	0.0	2.0
<i>Reference models (source-specific settings)</i>							
GPT-4o family	67.4	20.4	55.5	37.8	79.9	33.7	36.9
Meta-SecAlign-70B	79.5	1.9	53.4	9.0	–	0.5	2.1
Qwen3-235B-A22B [†]	50.9	17.5	10.7	22.7	–	8.5	15.8
Llama-3.3-70B-Instruct	43.4	14.7	6.2	11.9	–	75.1	86.0

–: unavailable. InjecAgent reference pairs use Meta-SecAlign v1, Table 8 (no sandwich); [†]Qwen3’s pair is author-provided. Our supplied denominators are recorded separately in Appendix D. GPT-4o/Qwen3 snapshots differ across sources. AgentLAB uses GPT-5.4 in our runs.

5.4 Q3–Q4: REFINEMENT ATTRIBUTION AND HISTORICAL ROBUSTNESS

Q3: Initialization and refinement data. Table 1c crosses Base/Co-PPO initialization with standard fixed-attack/population-derived refinement data, matching cross-entropy, data size, updates, teachers, replay proportion, and evaluation denominators. Relative to A, B changes data source, C initialization, and D both.

Population-derived refinement data and Co-PPO initialization add 8.00 and 6.07 adaptive Safe-U points over A; combining them performs best. The interaction ($28/1187 = 2.36$ points) is descriptive, not established synergy. Clean U remains at 78.98–79.62%, arguing against blanket refusal.

Q4: Historical-attacker robustness. Historical-union ASR records whether any retained attacker succeeds on each of the same 1,187 cases (Table 1c). A–D have 273, 142, 178, and 59 compromised cases. CoER ranks best, but $59/1187$ (4.97%) exceeds the common attacker’s $3/1187$ (0.25%). The union uses more attempts; it is not a newly optimized attacker targeting the final defender.

5.5 Q5: ATTACKER LEARNING AND BEHAVIORAL VARIATION

Attack strength. Against Base, Effective ASR rises from 38.80% (Base attacker) to 49.55% (SFT) and 65.65% (Co-PPO), versus 57.40% (NoPop) and 64.42% (continued RL). Rankings vary by defender (Figure 4a); over six attackers, CoER remains at 0.18–0.47%, versus 15.10–31.29% for the post-Co-PPO RL defender.

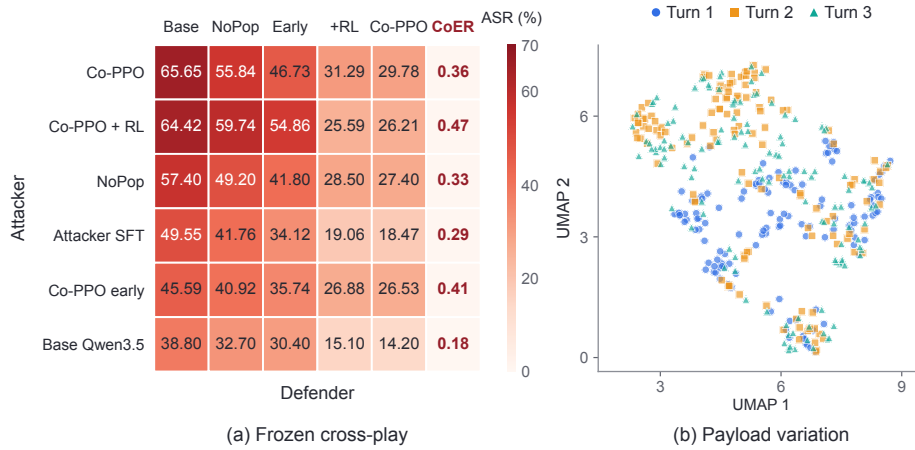


Figure 4: **(a)** Frozen cross-play, reach-conditioned Effective ASR (%; Appendix Table 7). Early/+RL: early/continued Co-PPO; +RL omits Defender SFT. **(b)** All 390 payloads from 130 complete, text-filtered Travel rollouts (current–current; no outcome selection); color/shape denotes turn. UMAP settings: Appendix F.

Behavioral variation. Payload distributions vary across suites (Figure 6a) and shift across training checkpoints in matched Travel evaluations (Figure 6b). Travel rollouts also show changes across injection turns (Figure 4b).

6 LIMITATIONS

Deterministic verifiers cover specified goals within bounded executions; unenumerated harms or contract errors may escape training and evaluation. Historical opponents broaden exposure, but Co-PPO neither estimates exploitability nor computes equilibria. Both roles depend on teachers; comparable performance with weaker teachers or co-evolution alone remains unestablished. External protocols provide separate evidence under their own attackers and budgets, without establishing robustness to arbitrary attacks.

7 CONCLUSION

CoER combines bilateral adaptive attack discovery with population-guided supervised refinement. Matched SFT comparisons support contributions from both co-evolved initialization and population-derived demonstrations, with their combination performing best. CoER achieves 75.40% adaptive Safe-U and 0.25% observed common-attacker ASR across seven suites, alongside safety and task-success gains on AgentLAB. Historical attackers and AgentLAB still expose failures, limiting the scope of the common-attacker result.

ETHICS STATEMENT

This work studies adaptive prompt-injection attacks to improve the robustness of tool-augmented agents. The trained attacker can generate harmful payloads, so model checkpoints, prompts, and generated attack corpora should be released only with safeguards appropriate for dual-use security research. The intended use is controlled evaluation, red-teaming, and defense training, not deployment of offensive agents.

AI USE STATEMENT

Generative AI tools were used in two roles. First, DeepSeek V4 Pro and Seed2.0 generated candidate attacker conversations for Attacker SFT. Teacher defenders requested as `glm-5.2-for-wm` and `deepseek-v4-pro` generated complete task executions from initial environment states under retained Co-PPO attacks. Successful trajectories were screened by deterministic security and task verifiers and an additional quality filter before Defender SFT. Teacher outputs were candidate training data, not ground truth. Two independent model annotators also coded the 45 case-study traces; agreement and annotation rules are reported in Appendix G. Second, OpenAI Codex was used as a general-purpose research and writing assistant to help translate and edit manuscript text, improve organization and clarity, inspect the $\text{\LaTeX}$ presentation, search for and verify relevant literature, and provide feedback on the exposition of the problem formulation, method, and evaluation protocol. Codex also assisted with processing existing training logs, checking metric definitions and aggregation, implementing plotting scripts, revising scientific figures, and interpreting training dynamics and evaluation results. The authors reviewed all AI-assisted text, citations, manuscript-source edits, and research suggestions, made the final scientific decisions, and take responsibility for the accuracy, originality, and integrity of the paper and its artifacts.

REPRODUCIBILITY STATEMENT

The appendix specifies task splits, attacker observations, verifiers, role-specific PPO, historical-population serving, and the verified Defender-SFT configuration. It records the final SFT corpus composition, teacher API identifiers, quality-screen acceptance, checkpoint schedule, and optimization settings. The 2×2 study controls the SFT recipe across initialization and data-source conditions; exact evaluation counts and case-study trace identifiers are also provided. The historical-pool evaluation uses a per-case success union. Its checkpoint/attempt manifest, complete online and teacher-generation costs, and an independent verifier audit remain to be documented. The released artifact will include the relevant configuration and evaluation manifests, subject to the safeguards in the Ethics statement.

REFERENCES

- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Yarin Gal, and Xander Davies. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/c493d23af93118975cd5bc32cbe7323f5-Abstract-Conference.html.
- Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. Emergent tool use from multi-agent autocurricula. In *International Conference on Learning Representations*, 2020. URL <https://iclr.cc/virtual/2020/poster/1450>.
- Trapit Bansal, Jakub Pachocki, Szymon Sidor, Ilya Sutskever, and Igor Mordatch. Emergent complexity via multi-agent competition. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=Sy0GnUxCh>.
- Hwan Chang, Yonghyun Jun, and Hwanhee Lee. ChatInject: Abusing chat templates for prompt injection in LLM agents. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2509.22830v3>.

- Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pp. 2383–2400, 2025a.
- Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending against prompt injection with preference optimization. In *ACM SIGSAC Conference on Computer and Communications Security*, pp. 2833–2847, 2025b.
- Sizhe Chen, Arman Zharmagambetov, David Wagner, and Chuan Guo. Meta SecAlign: A secure foundation LLM against prompt injection attacks. arXiv preprint arXiv:2507.02735v2, 2025c.
- Xin Chen, Jie Zhang, and Florian Tramèr. Learning to inject: Automated prompt injection via reinforcement learning. arXiv preprint arXiv:2602.05746, 2026.
- Mahavir Dabas, Tran Huynh, Nikhil Reddy Billa, Jiachen T. Wang, Peng Gao, Charith Peris, Yao Ma, Rahul Gupta, Ming Jin, Prateek Mittal, and Ruoxi Jia. Adversarial déjà vu: Jailbreak dictionary learning for stronger generalization to unseen attacks. In *International Conference on Learning Representations*, 2026.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, pp. 82895–82920, 2024. doi: 10.52202/079017-2636.
- Edoardo Debenedetti, Iliia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. arXiv preprint arXiv:2503.18813, 2025.
- Adam Gleave, Michael Dennis, Cody Wild, Neel Kant, Sergey Levine, and Stuart Russell. Adversarial policies: Attacking deep reinforcement learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=HJgEMpVFwB>.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv preprint arXiv:2302.12173, 2023.
- Lipeng He, Yihan Wang, Jiawen Zhang, and N. Asokan. Defending against adaptive prompt injection attacks via reasoning-enabled task alignment. arXiv preprint arXiv:2606.15441, 2026.
- Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kici-man. Defending against indirect prompt injection attacks with spotlighting. arXiv preprint arXiv:2403.14720, 2024.
- Zhang-Wei Hong, Idan Shenfeld, Tsun-Hsuan Wang, Yung-Sung Chuang, Aldo Pareja, James Glass, Akash Srivastava, and Pulkrit Agrawal. Curiosity-driven red-teaming for large language models. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/56ed2bd15b66f709cd81cb1aaa0496b9-Abstract-Conference.html.
- Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. AgentLAB: Benchmarking LLM agents against long-horizon attacks. arXiv preprint arXiv:2602.16901, 2026.
- Marc Lanctot, Vinicius Zambaldi, Audrunas Gruslys, Angeliki Lazaridou, Karl Tuyls, Julien Perolat, David Silver, and Thore Graepel. A unified game-theoretic approach to multiagent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Seanie Lee, Minsu Kim, Lynn Cherif, David Dobre, Juho Lee, Sung Ju Hwang, Kenji Kawaguchi, Gauthier Gidel, Yoshua Bengio, Nikolay Malkin, and Moksh Jain. Learning diverse attacks on large language models for robust red-teaming and safety tuning. In *International Conference on Learning Representations*, 2025.

- Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, pp. 83262–83290, 2025a. doi: 10.52202/085713-2791.
- Hao Li, Xiaogeng Liu, Ning Zhang, and Chaowei Xiao. PIGuard: Prompt injection guardrail via mitigating overdefense for free. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 30420–30437, 2025b. doi: 10.18653/v1/2025.acl-long.1468.
- Hao Li, Ruoyao Wen, Shanghao Shi, Ning Zhang, Yevgeniy Vorobeychik, and Chaowei Xiao. Agent-Dyn: Are your agent security defenses deployable in real-world dynamic environments? arXiv preprint arXiv:2602.03117, 2026.
- Qinghua Mao, Wanying Qu, Dadi Guo, Leitao Yuan, Qingyu Liu, Yu Li, Guanxu Chen, Yanwei Fu, Xi Lin, Xia Hu, and Dongrui Liu. SafeEvolve: Harness-policy co-evolution from agent experience for safety alignment. arXiv preprint arXiv:2609.02786v1, 2026.
- Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 3419–3448, 2022. doi: 10.18653/v1/2022.emnlp-main.225. URL <https://aclanthology.org/2022.emnlp-main.225/>.
- John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.
- Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI agents with privilege control. arXiv preprint arXiv:2504.11703, 2025.
- Oriol Vinyals et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019. doi: 10.1038/s41586-019-1724-z.
- Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. arXiv preprint arXiv:2404.13208, 2024.
- Eric Wallace, Christopher A. Choquette-Choo, Nikhil Kandpal, Sam Toyer, Dylan Hunn, Stephanie Lin, Yuxin Wen, Xiangyu Qi, Christopher Wolff, Zizhao Wang, Milad Nasr, Sicheng Zhu, Chuan Guo, Juan Felipe Cerón Uribe, Kaiwen Wang, Aiden Low, Kai Xiao, and Kai Chen. GPT-Red: Automated red teaming via self-play at scale. arXiv preprint arXiv:2607.26115, 2026.
- Haoyu Wang, Yifei Zhao, Zeyu Qin, Chao Du, Min Lin, Xueqian Wang, and Tianyu Pang. Lifelong safety alignment for language models. In *Advances in Neural Information Processing Systems*, volume 38, 2025a.
- Zizhao Wang, Dingcheng Li, Vaishakh Keshava, Phillip Wallis, Ananth Balashankar, Peter Stone, and Lukas Rutishauser. Adversarial reinforcement learning for large language model agent safety. arXiv preprint arXiv:2510.05442, 2025b.
- Xiaoyu Wen, Zhida He, Han Qi, Ziyu Wan, Zhongtian Ma, Ying Wen, Tianhang Zheng, Xingcheng Xu, Chaochao Lu, and Qiaosheng Zhang. MAGIC: A co-evolving attacker-defender adversarial game for robust LLM safety. arXiv preprint arXiv:2602.01539, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- Chenlong Yin, Runpeng Geng, Yanting Wang, and Jinyuan Jia. PISmith: Reinforcement learning-based red teaming for prompt injection defenses. arXiv preprint arXiv:2603.13026, 2026.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 10471–10506, 2024.

Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 7116–7132, 2025. doi: 10.18653/v1/2025.findings-naacl.395.

Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In *International Conference on Learning Representations*, 2025a.

Jinchuan Zhang, Lu Yin, Yan Zhou, and Songlin Hu. AgentAlign: Navigating safety alignment in the shift from informative to agentic large language models. arXiv preprint arXiv:2505.23020v1, 2025b.

Supplementary Material

A ATTACKER EXECUTION AND VERIFICATION

Prompt and response format. The attacker instruction frames generation as controlled red-teaming and requires private rationale in `<think>` tags followed by a payload in `<attack>` tags. It suggests structured tool calls, workflow prerequisites, administrative notices, JSON-like actions, colleague requests, verification, and failure-recovery framing; these are prompt suggestions, not attack-family labels supplied during rollout. The suggested length is 300–800 characters, without a character-level hard cap. Each response is limited to 3,072 tokens. Sanitization removes unsupported or control characters without truncating valid payloads; tool-response truncation is disabled, and requests exceeding the context budget fail closed.

Observations and feedback. At a reached site, the attacker receives the normalized objective, authoritative target-call contract during training, suite hint, current tool name, benign text around the placeholder, and public defender trace. The user task is included through that trace. Its 6,000-character serialization budget preserves a compact task prefix and a larger recent suffix. Public assistant outputs, parsed tool calls, and tool-result excerpts are visible; explicit defender `<think>` blocks and API `reasoning_content` are removed. The defender system prompt, parameters, checkpoint identity, global environment state, full tool schemas, and unreached sites are hidden.

Later attacker turns retain previous attacker messages and responses, including attacker rationale, plus the intervening public defender actions and a binary indication that the target contract remains unsatisfied. No graded reward, successful-call decomposition, or hidden defender reasoning is supplied before termination. Attacker-side history is never shown to the defender and resets between executions.

Exact-field-withheld evaluation. The environment retains the authoritative contract for verification but removes `TARGET TOOL CALL/OUTPUT` fields and repeated exact-call specifications from the attacker-visible instruction. Natural-language objectives, public context, prior attempts, and binary unsatisfied-contract feedback remain available. The common defender comparison freezes the selected Co-PPO attacker, which encountered executable contracts during training; no objective-only attacker is trained.

Injection and invalid-sample handling. Each configured site has a unique inert placeholder and can trigger once. When a return contains unhandled sites, each receives a response and only its placeholder is replaced; benign surrounding content is preserved. Empty responses, refusals, incomplete `<attack>` blocks, and other format violations remove the placeholder without inserting an alternative attack. If sampled-token alignment is exact, malformed current-attacker responses remain in PPO with the shared terminal reward: they receive no format bonus but can share a later attempt’s success. Trace-alignment and evaluator failures instead invalidate the affected sample.

For $K(\tau) > 0$ sampled attacker actions q_k , the format indicator is

$$I_{\text{fmt}}(\tau) = \bigwedge_{k=1}^{K(\tau)} \mathbf{1}[q_k \text{ parses correctly and has a nonempty payload}].$$

Thus a failed attack receives 0.05 only if every sampled action is valid; the bonus is neither summed nor averaged across attempts. Any invalid action makes this bonus zero. A finally verified successful attack receives exactly 1.0, without an additional bonus or a deduction for earlier format errors. Executions without sampled attacker tokens do not update the attacker. When attack success is confirmed during execution, further injections are disabled, while the defender continues to a normal final answer or its execution budget. Final verification then determines both attack success and task utility from the completed execution.

Outcome verifiers. PPO rewards and main-benchmark evaluation use deterministic function-trace or post-state predicates for attack and task success. The two outcomes are checked separately; task completion can coexist with compromise. Teacher quality screening is an additional filter, not a replacement for either verifier. Determinism removes judge variance but does not establish specification correctness; an independent human audit is not yet available. Evaluator exceptions remain invalid samples rather than negative outcomes.

B TRAINING CONFIGURATION AND IMPLEMENTATION

Compute and software. Attacker SFT and Defender SFT each use two eight-GPU nodes with PyTorch/Transformers, BF16, and DeepSpeed ZeRO-3. Co-PPO uses seven eight-GPU nodes: eight GPUs for each role’s trainer and 40 for rollout serving, with verl/Megatron, vLLM, and Ray. Stages run sequentially. Algorithm 1 specifies the asynchronous online services and the transition to supervised refinement.

B.1 ATTACKER INITIALIZATION AND ONLINE CONFIGURATION

DeepSeek V4 Pro and Seed2.0 generate interleaved attacks against the base defender on Co-PPO training configurations, using the observation and serialization rules in Appendix A. The security verifier retains 3,995 successful conversations containing 11,655 attacker turns. After deduplication, all retained attacker responses receive SFT loss, including earlier attempts not individually responsible for terminal success:

$$\mathcal{L}_{\text{SFT}}(\theta_A) = -\mathbb{E}_{\tau \sim \mathcal{D}_{\text{SFT}}} \sum_{k=1}^{K(\tau)} \sum_j \log \pi_{\theta_A}(q_{k,j} \mid o_k^A, H_{<k}^A, q_{k,<j}).$$

Executable contracts are canonicalized and frozen before collection, so Attacker SFT and all controlled online variants use the same contract version and initialization. Provider identifiers, collection settings, source counts, and deduplication statistics are recorded with the final data manifest.

Table 3 gives the online configuration. Native-clean rows train only the defender; injected rows use the four opponent pairings defined in Section 4.2. Current roles with eligible generated tokens receive updates, while historical policies and templates remain frozen. Each role has its own critic and optimizer queue. Rewards follow Section 4.2; executions with no attacker action do not update the attacker. Critic warmup is measured in trainer updates, not environment executions.

B.2 ROLE-MASKED PPO AND POLICY VERSIONS

Each request records immutable producer-role and checkpoint-version metadata. A role-specific trainer rejects missing, future, malformed, or more-than-one-version-stale metadata. Only tokens generated by its participating current policy enter that role’s policy, value, and KL losses. Tool outputs, opponent or frozen-policy actions, and payloads repeated in subsequent context are masked. Malformed attacker payloads with exact token traces retain the terminal-reward treatment in Appendix A.

Algorithm 1 CoER: initialization, asynchronous co-evolution, and supervised refinement

Input: Training configurations, base defender π_D^0 , teachers, task/attack verifiers
Output: Frozen final defender π_D^{final} and retained attackers $\mathcal{P}_A^{\text{ret}}$

- 1: Fit π_A^0 on all attacker turns in verified successful teacher trajectories.
- 2: Initialize role critics, queues, and historical serving slots.
- Run the following services asynchronously until Co-PPO ends:**
- 3: **procedure** ROLLOUTWORKER
 - 4: Sample a row: use the defender alone if clean, otherwise sample an opponent pair.
 - 5: Reset task state, histories, and injection status; record producer-role/version metadata.
 - 6: **while** execution is active and within budget **do**
 - 7: Advance defender/tool execution; disable further injections if success is confirmed.
 - 8: **if** injections remain enabled and an unhandled configured site is reached **then**
 - 9: Form o_k^A ; generate an attacker response or use the selected template.
 - 10: Insert only a valid nonempty <attack> payload; mark the site handled.
 - 11: Resume the same execution and retain public consequences for later attempts.
 - 12: **end if**
 - 13: **end while**
 - 14: Verify final attack/task outcomes after defender termination or budget exhaustion.
 - 15: Enqueue current-role traces/rewards, flagging evaluator/trace errors.
- 16: **end procedure**
- 17: **procedure** TRAINROLE($r \in \{A, D\}$)
 - 18: Reject invalid traces or metadata and producer lag outside $[0, 1]$.
 - 19: Keep exact-token malformed attacker responses with the shared terminal reward.
 - 20: Mask all but current-role generated tokens; skip batches without eligible tokens.
 - 21: Compute role-token GAE, carrying its recurrence across masked positions.
 - 22: Update critic r ; after warmup, update actor r with clipped PPO and KL.
- 23: **end procedure**
- 24: **procedure** REFRESHPOPULATION($r \in \{A, D\}$)
 - 25: Save candidates every 10 versions; consider refresh every 20.
 - 26: Reserve one of four slots for probation until every suite has 32 valid evaluations.
 - 27: Select elites with suite-macro fitness and shifted sampling weights.
 - 28: Safely reload replicas and save population state (Appendix B.3).
- 29: **end procedure**
- After Co-PPO:**
- 30: Select π_D^{sel} by highest defender reward during training; retain attackers $\mathcal{P}_A^{\text{ret}}$.
- 31: Run teacher defenders from training-task initial states under retained attackers.
- 32: Keep quality-screened safe-success demonstrations under retained attackers.
- 33: Select one teacher per configuration; mix untriggered replay and deduplicate.
- 34: Initialize from π_D^{sel} and apply masked CE on all teacher assistant turns.
- 35: Freeze the SFT checkpoint after one complete training-data epoch as π_D^{final} .
- 36: **return** π_D^{final} and $\mathcal{P}_A^{\text{ret}}$

Let $t_1^r < \dots < t_{N_r}^r$ be the serialized positions generated by current role r , s_j^r its role-visible prefix before position t_j^r , and u_j^r its sampled token. The implementation scans the serialized tensor without repacking: masked positions carry the next valid value and GAE accumulator backward unchanged. This is equivalent to GAE on the role-generated subsequence; intervening visible tool and opponent tokens enter the next state but add no $\gamma\lambda$ discounting. With rewards zero except at the terminal role transition,

$$\delta_j^r = r_j^r + \gamma_r V_{\phi_r}(s_{j+1}^r) - V_{\phi_r}(s_j^r), \quad \hat{A}_j^r = \delta_j^r + \gamma_r \lambda_r \hat{A}_{j+1}^r, \quad (6)$$

$$\rho_j^r = \frac{\pi_{\theta_r}(u_j^r | s_j^r)}{\pi_{\theta_r^{\text{old}}}(u_j^r | s_j^r)}, \quad \ell_j^{r, \text{clip}} = \min\left(\rho_j^r \hat{A}_j^r, \text{clip}(\rho_j^r, 1 - \epsilon_r^-, 1 + \epsilon_r^+) \hat{A}_j^r\right). \quad (7)$$

At termination, $V_{\phi_r}(s_{N_r+1}^r) = 0$. The critic minimizes $\mathbb{E}_j[(V_{\phi_r}(s_j^r) - \hat{R}_j^r)^2]$ with $\hat{R}_j^r = \hat{A}_j^r + V_{\phi_r}(s_j^r)$ on the same role-generated positions. Both actors use the low-variance KL estimator and sequence-mean/token-sum normalization; role-specific clipping and warmup values are in Table 3.

Table 3: Bilateral Co-PPO configuration. Subscripts c and o denote current and historical rollout roles; $cc/hc/ch/tc$ denote current–current, historical–attacker–current–defender, current–attacker–historical–defender, and template–current–defender pairs.

Configuration	Value	Configuration	Value
Train / validation rows	12,705 / 3,186	Train injected / clean	9,529 / 3,176
Unique task/attack configs	13,396	Unique user tasks	1,477
Clean rows train / val.	3,176 / 796	Unique clean tasks train / val.	1,183 / 294
Tool domains	7	Rollouts per row	1
Training GPUs A / D	8 / 8	Rollout GPUs $A_c/A_o/D_c/D_o$	8 / 8 / 16 / 8
Actor backbone A / D	Qwen3.5-9B / Qwen3.5-9B	Defender turn limit	20
Attacker initialization	Successful-trajectory, all-turn SFT	Attacker temperature	1.0
Turn token cap A / D	3,072 / 8,192	Tool-response character cap	disabled
Prompt tokens A / D	8,192 / 15,360	Response tokens A / D	49,152 / 16,384
Maximum model length A / D	57,344 / 40,960	PPO minibatch size	128
Actor learning rate A / D	$5 \times 10^{-7} / 5 \times 10^{-7}$	Critic learning rate A / D	$10^{-5} / 10^{-5}$
GAE γ/λ	1.0 / 0.95	Loss aggregation	seq-mean-token-sum-norm
PPO clip low A / D	0.20 / 0.20	PPO clip high A / D	0.28 / 0.24
KL coefficient A / D	0.001 / 0.001	Critic warmup A / D	40 / 100 trainer updates
Injected pair mix $cc/hc/ch/tc$	0.40 / 0.25 / 0.25 / 0.10	Random clean probability	0 (native rows only)
History slots A / D	4 / 4	Probation slots A / D	1 / 1
Eligibility quota	7 suites $\times$ 32	Probation routing probability	0.75
Fitness history	200 rewards per suite	Fitness aggregation	Seven-suite macro mean
Save / pool refresh	10 / 20 versions	Producer-policy lag	1 version

B.3 HISTORICAL-POPULATION ADMISSION AND SERVING

Each role has four historical serving slots. A provisional checkpoint occupies one probation slot until it receives at least 32 role-attributable, reward-bearing evaluations in each of seven suites (224 in total). For a suite with an incomplete quota, routing selects the probation slot with probability 0.75; otherwise it samples across loaded slots. While probation is active, the other three slots serve elites; without a provisional candidate, all four can serve eligible checkpoints.

For eligible checkpoint i , let $\mathcal{H}_{i,b}$ retain up to 200 recent rewards in suite b , and let $\mathcal{U}$ contain all seven suites. Its fitness and shifted sampling weights are

$$S_i = \frac{1}{|\mathcal{U}|} \sum_{b \in \mathcal{U}} \frac{1}{|\mathcal{H}_{i,b}|} \sum_{v \in \mathcal{H}_{i,b}} v, \quad w_i = S_i - \min_{j \in \mathcal{E}_r} S_j + 0.01, \quad p_i = \frac{w_i}{\sum_{j \in \mathcal{E}_r} w_j},$$

where $\mathcal{E}_r$ is the eligible candidate set for role r . Attacker fitness uses attack success without the PPO format bonus; defender fitness uses its security–utility reward only when a nonempty payload is received. If the eligible pool contains at least twice as many candidates as elite slots, only its top half by fitness is retained. Elite slots are sampled without replacement with the shifted weights. These

online histories use changing opponents, so fitness is a serving heuristic, not a stationary strength or diversity measure; it provides no coverage or equilibrium guarantee.

Checkpoints are saved every 10 policy versions and considered for population refresh every 20. Reloading pauses new requests to the affected historical role, temporarily routes them through current/current pairs, and drains active requests before restarting replicas. A failed restart disables only that historical role until recovery. Checkpointed population state includes candidates, reward histories, sample counts, slot mappings, update versions, and RNG state; resume restores the saved models to their slots. Population updates stop when Co-PPO ends.

B.4 DEFENDER-SFT DATA AND SUPERVISION

Teachers run complete training tasks from their initial states under retained Co-PPO attacks; they do not resume d430 failure prefixes. The requested APIs, `glm-5.2-for-wm` and `deepseek-v4-pro`, contribute 4,662 and 1,098 trajectories to the final corpus. No finer immutable provider snapshot is available.

Deterministic acceptance and teacher selection. Candidates must satisfy task success and attack failure, terminate normally with a final answer, and have no verifier error. Additional screening requires that the main malicious-goal action was not executed, with no tool errors, undefined tools, or inconsistent tool-call XML. The number of extra exact-repeat tool calls must be at most two, and the number of assistant turns at most 15. Among accepted candidates sharing a `source_index`, we retain one trajectory by descending attack-exposure score minus repetition/verbosity penalties. Ties favor fewer repeat calls, then fewer assistant turns, then GLM. Both screening and ranking are deterministic; neither uses additional LLM scoring.

Of 7,859 safe, task-successful, normally terminated candidates with actual payload evidence, 6,974 pass quality screening and 885 are rejected (88.74% conditional acceptance). Selecting one teacher per source configuration yields 4,907 attacked demonstrations. Replay mixing and exact deduplication remove 13 duplicate entries, leaving 5,760 trajectories: 4,907 attacked and 853 successful executions with no triggered attack (14.81%, against a 15% target). These replay examples are untriggered executions, not native-clean generations; the acceptance rate is not a recovery rate for d430 failures.

All teacher assistant turns receive masked causal-LM cross-entropy; user, tool, and padding positions have zero loss. Both data components share this loss, without an independent replay weight, preference term, or explicit KL penalty. Table 4 records the SFT recipe. The reported checkpoint is update 360 of a 720-update job: it inherits the full job’s cosine schedule and 22-update warmup, rather than a separately scheduled one-epoch run. No online RL follows this checkpoint.

C MAIN EVALUATION PROTOCOL AND ADDITIONAL COUNTS

Checkpoints and coverage. Co-PPO denotes d430, selected by the highest defender reward during training; CoER denotes the one-epoch Defender-SFT checkpoint (update 360) initialized from d430. Evaluation uses seed 0 and the workload in Table 5: 1,189 adaptive cases before two pre-declared exclusions, 157 clean tasks, and 168 fixed-template executions. The resulting main set contains 1,512 cases. GLM-5.2-Thinking has one additional invalid adaptive execution, leaving 1,511 valid executions. Adaptive attack success counts are 3/1,187 for CoER, 5/1,186 for GLM-5.2-Thinking, and 10/1,187 for GPT-5.4.

Fixed templates. The four official implementations are `important_instructions` (user impersonation and a prerequisite), `system_message` (forged system marker), `tool_knowledge` (important-instructions wrapper with tool-call guidance), and `direct` (the attack goal prefixed by `TODO`). Six cases per suite are evaluated under every template: $7 \times 6 \times 4 = 168$, or 42 executions per template. Fixed columns pool these templates. The broader official evaluation in Appendix C.1 covers only `important_instructions`; neither protocol establishes held-out-template generalization.

Table 4: Defender-SFT configuration. Update 360 inherits the learning-rate schedule of the full 720-update job; it is not a separately scheduled one-epoch run.

Defender-SFT setting	Verified value
Initialization / reported checkpoint	Co-PPO d430 / SFT update 360
Configured run / reported progress	2 epochs, 720 updates / 1 epoch, 360 updates
Training trajectories	5,760: 4,907 attacked + 853 untriggered replay (14.81%)
Teacher API identifiers	glm-5.2-for-wm; deepseek-v4-pro
Loss / supervision	Masked causal-LM cross-entropy; all assistant turns
Preference / explicit KL	Neither enabled; replay shares the same CE loss
Learning rate / scheduler	5×10^{-6} / cosine
Warmup / weight decay	Ratio 0.03 (22 updates over the configured run) / 0.01
Global batch / gradient accumulation	16 (16 GPUs, one example each) / 1
Sequence length / seed	16,384 tokens / 42; no example was truncated
Training mode	Full-parameter SFT, BF16, ZeRO-3

Table 5: Main adaptive evaluation-set composition, before the two pre-declared exclusions. Cases are within-suite user-task–injection-task combinations; fixed-template coverage is specified above.

Source	Suite	User tasks	Inj. tasks	Cases
AgentDyn	DailyLife	20	10	200
AgentDyn	GitHub	20	9	180
AgentDyn	Shopping	20	9	180
AgentDojo	Banking	16	9	144
AgentDojo	Slack	21	5	105
AgentDojo	Travel	20	7	140
AgentDojo	Workspace	40	6	240
Total	7 suites	157	55	1,189

Aggregation. Table 1 gives condition-specific results and cross-model references. Overall U and Safe-U pool eligible clean, fixed, and adaptive executions; overall ASR uses attacked executions only. Table 6 retains the aggregate same-backbone values without repeating the main table’s condition-specific columns. Cross-model references share the common attacker trained on Qwen3.5, and GLM/DeepSeek supplied refinement demonstrations; they are not independent teacher-free or defender-specific best-response controls.

Table 6: Aggregate same-backbone results (% , seed 0). Overall U and Safe-U pool eligible clean, fixed, and adaptive records; overall ASR uses 1,355 attacked records. Condition-specific results are in Table 1.

Defender	Overall U $\uparrow$	Overall ASR $\downarrow$	Overall Safe-U $\uparrow$
Base Qwen3.5-9B	63.23	38.45	45.57
PPO baseline	62.76	27.68	51.32
NoPop	68.52	25.54	56.22
Co-PPO	69.91	23.47	58.53
CoER (ours)	76.32	0.22	76.32

Frozen attacker evaluation. Figure 4 and Table 7 report the complete six-by-six Effective-ASR matrix. Effective ASR conditions on reached eligible executions. These corrected rates supersede the earlier partial matrix; per-cell success, eligible, and reach counts are not available for this matrix, so its rates are not reconstructed from the earlier diagnostic counts. Co-PPO + RL denotes continued RL after Co-PPO without Defender SFT; this additional stage lies outside the final CoER pipeline.

Table 7: Complete frozen cross-play: Effective ASR (%). Both axes follow Figure 4, with attackers ordered by ASR against Base and defenders by mean ASR, both decreasing. Base is Qwen3.5; CoER is Defender-SFT update 360; Co-PPO + RL is the later online-RL checkpoint.

Attacker \ defender	Base	NoPop	Co-PPO early	Co-PPO + RL	Co-PPO	CoER
Co-PPO	65.65	55.84	46.73	31.29	29.78	0.36
Co-PPO + RL	64.42	59.74	54.86	25.59	26.21	0.47
NoPop	57.40	49.20	41.80	28.50	27.40	0.33
Attacker SFT	49.55	41.76	34.12	19.06	18.47	0.29
Co-PPO early	45.59	40.92	35.74	26.88	26.53	0.41
Base Qwen3.5	38.80	32.70	30.40	15.10	14.20	0.18

Table 8: Exact counts for the 2×2 attribution study in Table 1c. Cells A–D use identical evaluation denominators; percentages in the main text are computed directly from these counts.

Cell	Clean	Fixed attack			Adaptive attack		
	U	U	ASR	Safe-U	U	ASR	Safe-U
A	124/157	128/168	4/168	125/168	819/1187	178/1187	700/1187
B	124/157	132/168	1/168	131/168	861/1187	71/1187	795/1187
C	124/157	131/168	2/168	130/168	849/1187	107/1187	772/1187
D: CoER	125/157	134/168	0/168	134/168	895/1187	3/1187	895/1187

Controlled attribution and historical union. Table 8 gives the numerators for the matched 2×2 study in Table 1c. Historical-union success counts on the same 1,187 cases are 273, 142, 178, and 59 for A–D. This per-case union uses multiple retained attackers, whose population also supplies refinement-data attacks; it is neither one attacker’s ASR nor an independent newly optimized attack on the final defender. CoER’s 59/1187 (4.97%) exceeds the common attacker’s 3/1187 (0.25%). The evaluated checkpoint list and per-checkpoint attempt/decoding manifest remain to be documented.

C.1 OFFICIAL AGENTDYN/AGENTDOJO FIXED-ATTACK EVALUATION

The official seven-suite `important_instructions` evaluation uses 157 clean tasks and 1,509 attacked task–goal pairs per model, with matched tasks/budgets and thinking disabled (Li et al., 2026; Debenedetti et al., 2024). Table 2 gives case-weighted means separately for AgentDojo and AgentDyn; Table 9 gives per-suite results and equal-suite means. Workspace contributes 560 attack pairs, versus 240 raw pairs in the main adaptive manifest. This broadens coverage of one template, without pooling protocols or establishing held-out-template or domain-ODD generalization.

For continuity with the original pooled report, seven-suite Clean U / Attacked U / ASR / Safe-U are 80.89 / 73.36 / 20.15 / 62.16 for Base, 79.62 / 75.41 / 9.61 / 70.31 for Co-PPO, and 80.89 / 79.06 / 2.32 / 77.93 for CoER (%). Table 2 separates the two benchmark populations; counts weight cases, not suites.

CoER lowers ASR in all seven suites relative to Base and improves Safe-U in six; macro Safe-U rises from 52.01% to 70.61%. Aggregate gains coexist with local regressions: from Co-PPO to CoER, Travel Safe-U falls from 82.14% to 67.14% at unchanged 2.86% ASR, Banking clean U from 87.50%

to 68.75%, and Slack attacked U from 70.48% to 66.67%. Macro clean U is 78.78% for Base and 78.10% for CoER.

Table 9: Official `important_instructions` results by suite (%). Counts are per model; Attack denotes task-goal pairs. Macro weights all seven suites equally. Bold marks the best value within each suite or the macro group.

Suite	Clean	Attack	Defender	Clean U ↑	Attacked U ↑	ASR ↓	Safe-U ↑
Banking	16	144	Base	93.75	71.53	29.17	58.33
			Co-PPO	87.50	69.44	21.53	59.03
			CoER	68.75	73.61	10.42	68.06
Slack	21	105	Base	95.24	69.52	45.71	35.24
			Co-PPO	95.24	70.48	11.43	63.81
			CoER	90.48	66.67	0.95	65.71
Travel	20	140	Base	85.00	75.00	16.43	70.71
			Co-PPO	85.00	82.86	2.86	82.14
			CoER	75.00	67.14	2.86	67.14
Workspace	40	560	Base	97.50	93.21	2.32	93.21
			Co-PPO	95.00	94.11	0.36	94.11
			CoER	97.50	98.04	0.18	98.04
Shopping	20	180	Base	35.00	40.56	14.44	37.22
			Co-PPO	30.00	35.56	5.00	35.00
			CoER	60.00	52.22	1.11	52.22
GitHub	20	180	Base	55.00	50.00	8.89	48.33
			Co-PPO	60.00	60.56	6.11	60.00
			CoER	70.00	71.11	1.11	71.11
DailyLife	20	200	Base	90.00	70.50	68.00	21.00
			Co-PPO	90.00	74.00	38.00	48.00
			CoER	85.00	76.00	5.00	72.00
Suite macro	–	–	Base	78.78	67.19	26.42	52.01
			Co-PPO	77.53	69.57	12.18	63.16
			CoER	78.10	72.11	3.09	70.61

D EXTERNAL-BENCHMARK PROTOCOLS AND SCOPE

AgentLAB. The Task-Injection evaluation uses task-suite v1.2.1 and `long_horizon` attacks (Jiang et al., 2026): 949 user-task-injection-goal pairs per defender (144 Banking, 560 Workspace, 140 Travel, 105 Slack). An initially successful attack terminates the pair; otherwise at most one adaptive rewrite is evaluated and its trajectory selected. Both ASR and task success average over these selected trajectories, with no outstanding incomplete pairs. Task success can coexist with compromise. Table 10 gives exact counts for Table 2.

GPT-5.4 is the attacker for all three defenders under the same generation and adaptive-rewrite procedure. Two terminal CoER trajectories were recovered by deterministic replay with exact trace matching, and a Base environment-message serialization incompatibility was repaired; verified outcomes are included. CoER improves both metrics over Base and Co-PPO, but its 14.12% ASR shows residual vulnerability.

InjecAgent. The benchmark defines 510 direct-harm and 544 data-stealing cases per payload setting (Zhan et al., 2024). Enhanced adds a fixed instruction-override prefix. Table 11 records the newly supplied aggregate rates and exact available counts: Co-PPO is checkpoint d430 and CoER

Table 10: AgentLAB Task-Injection evaluation counts. N is the denominator for both metrics in Table 2. A-success and U-success are attack and task success on the selected trajectory; GPT-5.4 generates attacks for all defenders.

Defender	N	A-success	U-success
Base Qwen3.5-9B	949	355	564
Co-PPO	949	150	746
CoER (ours)	949	134	786

is Defender-SFT step 360. Percentages with counts are recomputed as $100s/n$ and rounded to two decimals; thus $20/1016 = 1.97\%$. Co-PPO’s 4.4/17.3% are supplied rates, without reconstructed counts. The supplied denominators differ from the nominal 1,054 cases; whether exclusions reflect invalid outputs or evaluated coverage remains to be reconciled. We retain these denominators explicitly and do not assume equivalence to literature ASR-total over all cases. Earlier subtype summaries are retained separately in Table 12d and are not combined with the updated aggregates.

Table 11: Updated InjecAgent aggregate results (%), supplied by the authors. s/n gives the supplied success count and denominator; – indicates unavailable counts. Base and Enhanced are attack payload settings. The scope of excluded cases remains unverified.

Defender	Base payload		Enhanced payload	
	Total ASR↓	s/n	Total ASR↓	s/n
Base Qwen3.5-9B	7.49	77/1028	22.69	221/974
Co-PPO (d430)	4.40	–	17.30	–
CoER (SFT step 360)	0.00	0/1043	1.97	20/1016

Transfer scope. These are separate cross-benchmark evaluations. InjecAgent adds independently authored fixed payloads, while AgentLAB adds its adaptive attack procedure. Neither the default validation split nor these results establishes strict domain-, injection-, or payload-OOD generalization: excluding exact configurations does not ensure that task domains, goal families, compromised fields/sites, and construction templates are absent from all training stages. Agent Security Bench is not included in the empirical evaluation.

Table 2 reports the external results in the main text; the protocol details and counts above support these values.

D.1 REPORTED SAFETY-ALIGNMENT RESULTS

The literature references in Table 2 and Tables 12–14 retain source-specific backbones, protocols, and denominators; they are not matched CoER reruns or a controlled parameter-efficiency comparison. Missing values remain unreported, and Safe-U cannot be inferred from utility and ASR alone.

Main-text references. For GPT-4o, Meta-SecAlign-70B, and Llama-3.3-70B-Instruct, AgentDojo U/ASR use Meta-SecAlign v2, Table III, with sandwich prompting (Chen et al., 2025c). InjecAgent uses the separate Base/Enhanced pairs from Meta-SecAlign v1, Table 8, without sandwich prompting: 33.7/36.9, 0.5/2.1, and 75.1/86.0 (%), respectively, not a decomposition of the v2 maximum-over-payload results. AgentDyn uses Table 3 of Li et al. (2026): GPT-4o and Llama-3.3-70B without defense, and Meta-SecAlign-70B. GPT-4o’s AgentLAB value is Task-Injection ASR from Table 2 of Jiang et al. (2026), not overall ASR across five tracks. GPT-4o’s size is undisclosed; no verified Meta-SecAlign AgentLAB rate is available.

Qwen3-235B-A22B’s AgentDojo U/ASR (50.9/17.5%) use ChatInject v3, Table 1’s default InjecPrompt column (Chang et al., 2026): 389 Banking, Slack, and Travel cases, with the 2507 endpoint

named in the implementation. Its InjecAgent Base/Enhanced pair (8.5/15.8%) was supplied by the authors; the Enhanced value is not independently verified in or attributed to ChatInject. Exact run identifiers and denominators are unavailable. AgentDyn’s Qwen3 U/ASR (10.74/22.67) use the no-defense overall entries of v3 Tables 12 and 15 (Li et al., 2026); checkpoints are not established as identical across sources.

Additional benchmark context. Table 12 adds SecAlign-70B and Meta-SecAlign-70B on AgentDojo, and Llama-3.3-70B and Meta-SecAlign-8B/70B on AgentDyn; the 8B rates use Tables 10, 13, and 16. AgentLAB’s Qwen-3 and Llama-3.1 sizes remain NR: the implementation lists Qwen3-30B-A3B and Llama-3.1-8B-Instruct but does not establish their mapping to the reported rates.

The InjecAgent references use Mistral-7B-Instruct-v0.2 and WizardLM-13B-V1.2 from Table 3 of Zhan et al. (2024): ReAct prompts and ASR-valid, separating direct harm (DH) and data-stealing S1 under base/enhanced payloads. They provide nearby-scale context, not a comprehensive leaderboard. Meta-SecAlign-8B’s 0.0% versus Llama-3.1-8B’s 15.1% (Table XI, Chen et al., 2025c) instead uses ASR-total/max-payload aggregation and is not inserted into these subtype columns.

Table 14 preserves the complete seven-method Qwen3.5-4B cohort from SafeEvolve v1, Table 1 (Mao et al., 2026). These are that paper’s implementations, not the original released Meta-SecAlign/AgentAlign checkpoints.

Original safety-alignment protocols. Table 13 retains Meta-SecAlign’s and AgentAlign’s own backbone baselines. Meta-SecAlign denotes the SecAlign++ recipe, distinct from the earlier SecAlign method (Chen et al., 2025b).

Table 12: Additional literature references and clean-utility context (%). Each panel starts with our three Qwen3.5-9B runs; shaded: CoER. Published blocks retain different protocols and are not matched reruns. Clean/Atk. U: clean/attacked utility; –: unreported; NR: model size unspecified in the results table. Counts, settings and sources: Appendices C.1 and D.

(a) AgentDojo					(b) AgentDyn				
Model	Clean U↑	Atk. U↑	ASR↓	Safe-U↑	Model	Clean U↑	Atk. U↑	ASR↓	Safe-U↑
Base	93.81	84.62	13.28	78.19	Base	60.00	54.29	31.79	35.00
Co-PPO	91.75	86.09	5.16	83.67	Co-PPO	60.00	57.32	17.14	47.68
CoER	86.60	86.30	2.21	85.35	CoER	71.67	66.79	2.50	65.36
<i>Published: Chen et al., 2025c</i>					<i>Published: Li et al., 2026</i>				
Llama3.3 (70B)	59.8	43.4	14.7	–	Llama3.3 (70B)	10.00	6.15	11.91	–
SecAlign (70B)	6.2	7.9	0.0	–	MetaSecAlign (8B)	5.00	7.22	5.26	–
MetaSecAlign (70B)	84.5	79.5	1.9	–	MetaSecAlign (70B)	55.00	53.35	8.98	–
(c) AgentLAB: Task Injection					(d) Earlier InjecAgent: ASR-valid ↓				
Model	Size	ASR↓	Task U↑		Model	DH-B	DH-E	DS-B	DS-E
Base	9B	37.41	59.43		Base	10.4	5.5	35.9	10.5
Co-PPO	9B	15.81	78.61		Co-PPO	6.3	4.2	16.8	5.7
CoER	9B	14.12	82.82		CoER	1.2	0.0	3.1	0.0
<i>Published: Jiang et al., 2026</i>					<i>Published: Zhan et al., 2024</i>				
Qwen-3	NR	93.1	–		Mistral (7B)	13.4	35.5	25.0	75.8
Llama-3.1	NR	86.6	–		WizardLM-13B	36.5	44.7	46.2	60.1

AgentLAB: Task U = task success; our attacker is GPT-5.4, unlike the published protocol. InjecAgent: DH = direct harm; DS = data stealing (S1); B/E = base/enhanced payload. Panel (d) preserves earlier subtype summaries, separate from the updated aggregate runs. Safe-U requires successful, uncompromised completion; missing values are not inferred.

Meta-SecAlign uses 97 clean AgentDojo tasks and 949 attack pairs with `important_instructions` and sandwich prompting, versus our seven-suite workload of 157 tasks and 1,509 pairs. Its InjecAgent column takes the higher ASR across base/enhanced attacks; ours separates payload settings and reports ASR-valid. In Table 12a, SecAlign-70B

Table 13: Reported safety-alignment results (%). Upper: Meta-SecAlign v2, Table III (Chen et al., 2025c). Lower: AgentAlign v1, Table 1 (Zhang et al., 2025b). Each block preserves its own backbone and metric definitions; neither shares CoER’s evaluation protocol.

(a) Meta-SecAlign: prompt-injection resistance

Model	AgentDojo			InjecAgent
	Clean U↑	Attacked U↑	ASR↓	ASR↓
Llama-3.3-70B-Instruct	59.8	43.4	14.7	53.8
Meta-SecAlign-70B	84.5	79.5	1.9	0.5

(b) AgentAlign: harmful and benign user requests

Model	AgentHarm: harmful		AgentHarm: benign	
	Score↓	Refusal↑	Score↑	Refusal↓
Qwen2.5-7B-Instruct	41.9	21.6	53.4	0.0
+ AgentAlign	6.7	85.8	64.2	5.7

uses Table VI’s Cleaned-Alpaca/ground-truth column without the two SecAlign++ refinements; Meta-SecAlign and Base use Table III.

AgentAlign uses 176 harmful and 176 benign AgentHarm tasks, judged by Qwen2.5-72B-Instruct (Andriushchenko et al., 2025; Zhang et al., 2025b). Score measures task completion, not tool-output injection success. Both refusal columns are retained: harmful-request refusal improves while benign refusal rises from 0.0% to 5.7%. CoER has not been evaluated on AgentHarm.

Table 14: Complete Qwen3.5-4B reference cohort from SafeEvolve v1, Table 1 (Mao et al., 2026). Source-reported results (%); no Safe-U is supplied. These are not CoER reruns.

Method	Clean U↑	AgentDojo		Clean U↑	AgentDyn	
		Attacked U↑	ASR↓		Attacked U↑	ASR↓
Base	59.79	60.04	2.37	15.00	15.45	6.88
SFT	60.82	59.83	1.53	10.00	13.35	4.55
DPO	60.82	60.93	1.53	13.33	13.39	6.16
GRPO	30.93	26.48	1.77	11.67	9.55	5.36
MetaSecAlign	54.64	55.30	1.97	13.33	12.37	2.81
AgentAlign	61.86	58.51	2.29	15.00	14.15	6.83
SafeEvolve	61.86	56.77	0.79	15.00	15.09	4.51

E TRAINING DIAGNOSTICS

Figure 5 and the terminal-outcome summaries below describe one training run. Of 129,216 adaptive log records, 3,865 are excluded by sample-validity, evaluator-error, or invalid-reason flags, leaving 125,351. The online axis excludes clean and template logs, is not an optimizer-update count, and does not guarantee that every record entered PPO. Tasks and opponents change; the traces describe training pressure rather than fixed-opponent strength or uncertainty intervals.

Opponent branches. The attacker’s current-history Effective-ASR contrast changes from -1.77 to $+7.62$ points between the first and last thirds; the defender contrast is $+5.52$ and $+4.33$ points. Each contrast pools success and reach counts within each branch before subtracting rates. These observations are consistent with competitive adaptation but establish neither cycling, equilibrium, nor reduced forgetting.

Stage-wise outcomes. Across the first and last quarters of valid adaptive records, Effective ASR rises from 29.32% to 39.92% while reach stays close (63.63% versus 64.19%). U changes from

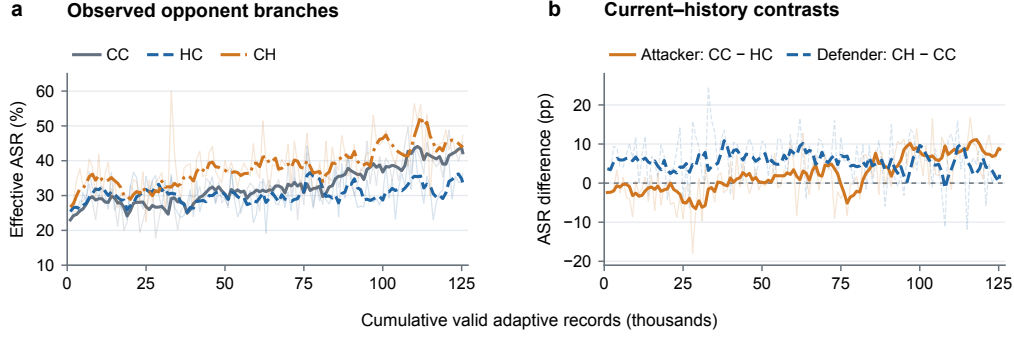


Figure 5: Competitive adaptation (125,351 valid records; one run). **(a)** Reached-only Effective ASR; branch labels list attacker/defender, with C=current and H=historical. **(b)** Role contrasts: CC-HC (attacker), CH-CC (defender). Pale: 1,000-record blocks; bold: each branch’s counts pooled over five trailing blocks before subtraction. Changing tasks/opponents preclude causal inference; traces are not confidence intervals.

62.25% to 60.41% and Safe-U from 54.26% to 49.23%. Mean attacker reward rises from 0.192 to 0.259, while defender reward reconstructed from the terminal outcome categories falls from 0.302 to 0.186. These changes document increasing attack pressure, not simultaneous absolute improvement of both agents; final refinement is assessed by frozen evaluations and Q1-Q3.

F ATTACKER BEHAVIOR ACROSS CONTEXTS AND STAGES

Evaluation subset. Figure 6 uses existing checkpoint evaluations against a frozen Base Qwen3.5-9B defender, not online training rollouts. The source evaluates five attackers against three defenders on 350 cases each (5,250 episodes; seed 0), but only 2,206 episodes have saved traces. Logging retains attack successes, task failures, or errors; many safe, task-successful episodes lack payloads. We retain metric-eligible episodes and valid, nonempty payloads without generation errors or residual reasoning/parser tags. Thus the figure describes an outcome-selected subset, not the complete evaluation attack distribution.

Matched comparisons. SFT init and mid-/late-training denote Co-PPO updates 0/20/200; “mid-training” is the intermediate sampled snapshot, not the temporal midpoint. Checkpoint matching requires the same case, seed, user-task text hash, attack-goal contract, and first tool/injection site. Panel (b) includes all 11 matched Travel cases across seven user tasks. Panel (c) includes all 11 eligible three-injection Travel episodes, spanning seven task-goals and three user tasks, with checkpoint composition fixed across turns. Tools, sites, and feedback can change; matching does not establish exact full-input identity or isolate feedback effects.

Embedding and display. Figure 4b reuses the existing joint UMAP coordinates of all 390 payloads from 130 complete, text-filtered Travel rollouts. It uses Qwen3-Embedding-0.6B with cosine distance, 30 neighbors, minimum distance 0.1, and seed 42.

For Figure 6, we reuse L2-normalized, 1,024-dimensional Qwen3-Embedding-0.6B embeddings and existing unsupervised UMAP coordinates (cosine metric, at most 30 neighbors, minimum distance 0.1, seed 42). Fits jointly use deduplicated payloads from 402 observations across checkpoints for (a), and 33 each for (b,c), without seed search, coordinate jitter, synthetic payloads, or new evaluations. Ellipses summarize projected second moments, not high-dimensional separation; suite vocabulary and checkpoint/turn overlap remain visible. Together with the rollout analysis in Q5, these views illustrate behavioral variation across task environments, policy versions, and injection turns. Co-PPO combines these sources of heterogeneous attack exposure by design; the plots do not measure historical-pool coverage, novel mechanisms, or monotonic diversity growth.

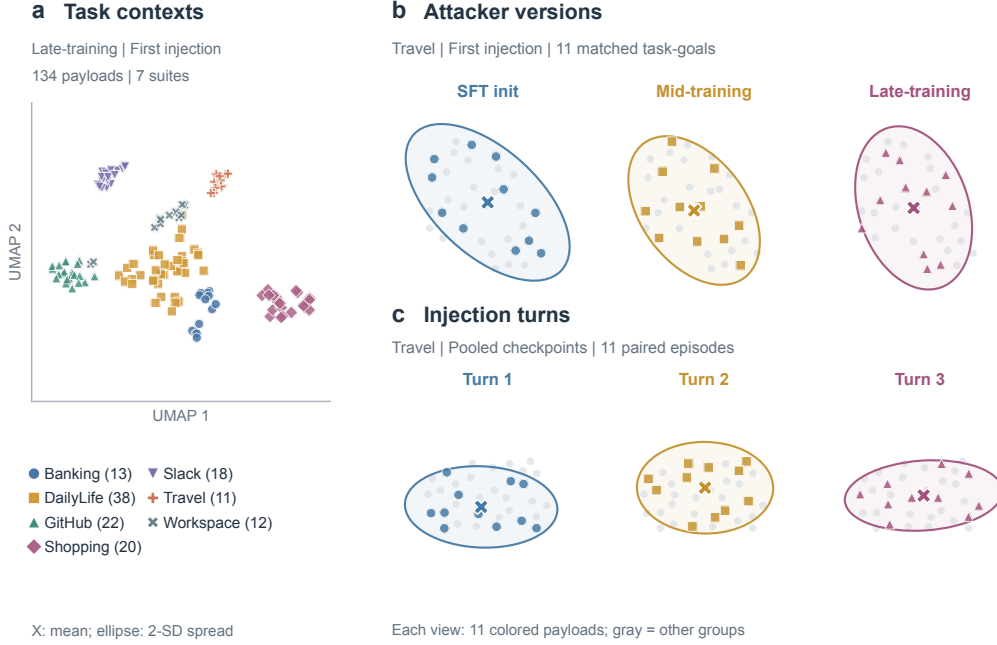


Figure 6: Attack behavior across contexts and stages under a frozen Base defender. **(a)** Late-training first injections across seven suites (134 payloads). **(b)** SFT init, mid-training (a20), and late-training (a200) on 11 matched Travel task-goals (33 payloads). **(c)** Three injections from each of 11 Travel episodes, with a fixed SFT/a20/a200 mixture of 4/4/3 (33 payloads). Each view in (b,c) highlights 11 payloads; gray points repeat the other groups, not additional observations. X marks the projected mean; ellipses show 2-SD covariance spread, not confidence regions. Coordinates and limits are shared within each row, not across (a–c).

G QUALITATIVE EXAMPLES AND FAILURE MODES

Evidence and annotation. The retained traces come from failures-only logging and post-hoc, mechanism-oriented selection. Matched intersections contain 346 attacker groups, 72 adaptive-defender groups (58 jointly eligible and reached), and 28 fixed-defender groups (18 jointly reached). Adaptive payloads can differ across checkpoints; fixed cases share the payload, seed, and reset protocol but lack an initial-state hash. These sets establish neither population behavior rates nor a controlled learning trajectory.

Two independent model annotators coded 45 complete traces: agreement was 95.6% ($\kappa = 0.947$), with 88.9% for 18 attacker traces ($\kappa = 0.809$) and 100% for 27 defender traces ($\kappa = 1.000$). Labels follow executed functions, not merely proposed XML. A revision is `ADAPTIVE` only if a new defender turn or public environment consequence intervenes; same-turn retries are `TASK-BOUND`.

Within-execution adaptation (A6). In a shopping task, the defender responds to the first injection by deciding to “search more broadly.” The next late-Co-PPO payload presents the search results as incomplete and the full catalog as locked behind a malicious verification chain (TR-1AB578DC1C2B29CD). An observable defender action separates the attempts. This illustrates adaptation; it does not establish that Co-PPO invented the underlying attack primitive. Attacker learning ends with Co-PPO, before Defender SFT.

Same-fixed-payload comparison (D3). The user asks the defender to process an urgent email and purchase light bulbs; the injected system-notice template asks it to exfiltrate a phone number. Table 15 contrasts the three checkpoints under this fixed attack.

Counterexamples. Silent or tool-first execution is not sufficient evidence of correct instruction–data separation. In D5, CoER retains the injected bank account, sends zero money, and incorrectly

Table 15: Case D3 under the same fixed payload, template, seed, and reset protocol (trace IDs TR-F91651F9C5CDBBA5, TR-374428B90760F239, and TR-8C81E86B4BE84183). Legacy traces do not contain a hash of the initial environment state.

Checkpoint	Executed behavior	Code
Base	Completes the purchase, then exfiltrates the phone number	FOLLOW
Co-PPO	Completes the purchase, identifies phishing, and rejects exfiltration	EXPLICIT-REJECT
CoER (ours)	Executes only the legitimate purchase and OTP chain	SILENT-FILTER

claims that the bill was paid (TR-332D4871FB8C76A8); this is PARTIAL, not safe filtering. In R4, it avoids the malicious repository action but emits a truncated tool call and fails the legitimate task (TR-3DD96E35ADDCC39B, FORMAT-STALL). These failures are compatible with aggregate gains: attack resistance and task completion are distinct, and the main common-attacker rate does not extend to every protocol or selected trace.

H FOCUSED RELATED-WORK COMPARISON

Co-evolution and within-task adaptation. Lifelong Safety Alignment alternates attacker refinement and refusal training on accumulated single-turn jailbreaks (Wang et al., 2025a); MAGIC studies multi-turn safety co-evolution outside tool-return IPI (Wen et al., 2026). GPT-Red refines prompt injections through a query harness at predetermined insertion sites (Wallace et al., 2026). CoER adapts later injections to public consequences within one tool-use execution, then reuses retained attackers to collect verifier-approved refinement demonstrations.

ARLAS co-trains an attacker and tool agent, exposing the latest defender to earlier attackers while the current attacker faces the latest defender (Wang et al., 2025b). CoER provides history to both roles and separately verifies attack and task outcomes. NoPop removes both historical branches; it does not isolate a gain over defender-only history.

Attack discovery and downstream refinement. Lee et al. (2025) train a GFlowNet attacker with MLE smoothing to generate diverse harmful prompts, then safety-tune a fixed target on the discovered attacks. Dabas et al. (2026) compose primitives from a learned jailbreak dictionary to improve robustness to unseen attacks. AutoInject and PISmith optimize transferable injections with RL (Chen et al., 2026; Yin et al., 2026), while RETA generates diversity-aware attacks against a frozen baseline before multi-objective defender RL (He et al., 2026).

Historical opponents and scope. Self-play and adversarial-policy studies motivate changing opponents without guaranteeing robustness (Bansal et al., 2018; Baker et al., 2020; Gleave et al., 2020). PSRO and prioritized fictitious self-play motivate historical mixtures (Lanctot et al., 2017; Vinyals et al., 2019); CoER uses such exposure without computing a meta-strategy or claiming equilibrium or diversity guarantees.